



Evolutionary Test Generation with Diversity-Aware Mutation Fitness

CS453 Team 6

Background: EvoSuite



<https://www.evosuite.org/>

- What is Evosuite?
 - Open-source tool for SBST
 - Automatically generates JUnit test suites
- How EvoSuite Generates Tests
 - Utilizes an evolutionary algorithm
 - Uses various factors to evaluate **fitness function**
 - Branch distance
 - Code coverage
 - **Mutation score**
 - Test suite properties
 - etc.

Motivation

- Mutation score is based on 'total killed mutants'.
- However, there are many operator types.
- Let M_1, M_2, M_3 be counts of killed mutants grouped by operator type.
 - For example:
 - M_1 - arithmetic mutations.
 - M_2 - comparison mutations.
 - M_3 - condition negation.

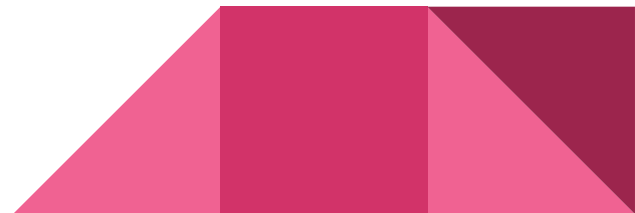


Motivation

- Consider two test suites with following mutation score:

Test Suite	Killed mutants	Mutation Score
A	$M_1=100, M_2=10, M_3=10$	Same
B	$M_1=40, M_2=40, M_3=40$	Same

- Which one is better?



Mutation Score as a Fitness Function?

- Some operator types are easier to kill early than others
- Mutation score alone can favor early, easy wins
- Naïve mutation score can create a “blind-spot”
 - The GA may over-invest in easy types and under-explore harder ones

Test Suite	Killed mutants	Mutation Score
A	$M_1=100, M_2=10, M_3=10$	Same
B	$M_1=40, M_2=40, M_3=40$	Same

Research Hypothesis

Coupling operator-type diversity into fitness guides early selection toward broader exploration — improving fault-model coverage without sacrificing mutation score.

Approach: Diversity Metric

- Use Shannon entropy over killed-mutant distribution

Diversity metric

Let c_i be killed mutants of operator type i , $p_i = c_i / \sum_j c_j$, and n the number of operator types in the class:

$$v = - \sum_{i=1}^n p_i \log p_i, \quad \hat{v} = \frac{v}{\log n} \in [0, 1], \quad \delta = 1 - \hat{v}$$

(from project's README.md)

- Nine operator types are tracked
- 

Approach: Coupling Diversity into Fitness

- Choose various candidates
- All having our custom parameter k

Fitness function

EvoSuite minimises fitness (0 = all mutants killed):

$$F_{\text{base}} = f_{\text{branch}} + \sum_{m \in \text{alive}} d_{\text{min}}(m)$$

Mode	Formula
NONE	F_{base}
MULTIPLICATIVE	$F_{\text{base}} \cdot (1 + k\delta)$
ADDITIVE	$F_{\text{base}} + k\delta$
EXPONENTIAL	$F_{\text{base}} \cdot e^{k\delta}$

(from project's README.md)

Implementation

- Cloned latest Evosuite from GitHub
- Manually injected our fitness score into Evosuite source code

```
...client/src/main/java/org/evosuite/coverage/mutation/StrongMutationSuiteFitness.java
128 Set<Integer> touchedMutants = new LinkedHashSet<>();
129
130 @Override
131 public double getFitness(TestSuiteChromosome suite) {
132     // TODO: is it enough?
133     String[] operators = { ReplaceVariable.NAME, InsertOperator.NAME,
134         ReplaceConstant.NAME,
135         ReplaceSwitchOperator.NAME };
136     Double[] fitnessValues = new Double[operators.length];
137     Double fitnessSum = 0.0;
138
139     // logger.info("Fitness values for " + numMutantFitness.size() + " mutants");
140     for (Map.Entry<Mutation, Double> entry : numMutantFitness.entrySet()) {
141         Mutation mutant = entry.getKey();
142         Double fit = entry.getValue();
143         int type = 0;
144         for (int i = 0; i < operators.length; i++) {
145             if (mutant.getMutationName().startsWith(operators[i])) {
146                 type = i;
147                 break;
148             }
149         }
150         fitnessValues[type] = fit;
151         fitnessSum += fit;
152     }
153     int type = 0;
154     Double entropy = 0.0;
155
156     for (int i = 0; i < operators.length; i++) {
157         entropy = entropy + fitnessValues[i] / fitnessSum *
158             Math.log(fitnessValues[i] / fitnessSum);
159     }
160
161     // TODO: is it correct?
162     fitness = fitnessSum + entropy;
163 }
164
165 // fitness
166 double fitness;
```

```
evosuite/client/src/main/java/org/evosuite/Properties.java
353
354 // Added by CS453 Team 6
355 public enum DiversityCoupling {
356     NONE,
357     // F = F_base * (1 + k * (1 - vMut)), w/
358     MULTIPLICATIVE,
359     // F = F_base * k * (1 - vMut), w/
360     ADDITIVE,
361     // F = F_base * exp(k * (1 - vMut)), w/
362     EXPONENTIAL,
363     // F = F_base * (1 + min(k * (1 - vMut), diversity_cap)), w/
364     CAPPED_MULTIPLICATIVE
365 }
366
367 // Added by CS453 Team 6
368 public static DiversityCoupling DIVERSITY_COUPLING = DiversityCoupling.NONE;
369
370 // Added by CS453 Team 6
371 @DoubleValue(min = 0.0, max = 1000.0)
372 public static double DIVERSITY_K = 0.0;
373
374 // Added by CS453 Team 6
375 @DoubleValue(min = 0.0, max = 1000.0)
376 public static double DIVERSITY_CAP = 1.0;
377
378 @Parameter(key = "algorithm", group = "Search Algorithm", description = "Search
379 algorithm")
380 public static Algorithm ALGORITHM = Algorithm.DYNAMOSA;
381
382 // Added by CS453 Team 6
383 private static int operatorBucketsOf(Mutation mutant) {
384     String name = mutant.getMutationName();
385     for (int i = 0; i < MUTATION_OPERATORS.length; i++) {
386         if (name.startsWith(MUTATION_OPERATORS[i])) {
387             return i;
388         }
389     }
390     return MUTATION_OPERATORS.length; // OTHER
391 }
392
393 // Added by CS453 Team 6
394 private double computeDiversityEfficiency(Set<Integer> newKilled) {
395     for (Map.Entry<Integer, MutationTestFitness> entry :
396         this.mutantMap.entrySet()) {
397         Integer id = entry.getKey();
398         Integer operatorBucket = operatorBucketsOf(entry.getValue().getMutation());
399         if (operatorBucketsPresent == 0) {
400             Set<Integer> present = new LinkedHashSet<>();
401             operatorBucketsPresent = Math.min(present.size(), 1);
402         }
403         Set<Integer> killed = new LinkedHashSet<>();
404         killed.addAll(newKilled);
405         long[] counts = new long[MUTATION_BUCKETS];
406         long total = 0;
407         for (Integer id : killed) {
408             Integer bucket = mutantTypeCache.get(id);
409             if (bucket != null) {
410                 counts[bucket]++;
411                 total++;
412             }
413         }
414         if (total == 0) {
415             operatorBucketsPresent = 1;
416             this.lastEntropy = 0.0;
417             this.lastEntropyMore = 0.0;
418             return 0.0;
419         }
420     }
421 }
```

```
evosuite/client/src/main/java/org/evosuite/Properties.java
353
354 // Added by CS453 Team 6
355 public enum DiversityCoupling {
356     NONE,
357     // F = F_base * (1 + k * (1 - vMut)), w/
358     MULTIPLICATIVE,
359     // F = F_base * k * (1 - vMut), w/
360     ADDITIVE,
361     // F = F_base * exp(k * (1 - vMut)), w/
362     EXPONENTIAL,
363     // F = F_base * (1 + min(k * (1 - vMut), diversity_cap)), w/
364     CAPPED_MULTIPLICATIVE
365 }
366
367 // Added by CS453 Team 6
368 public static DiversityCoupling DIVERSITY_COUPLING = DiversityCoupling.NONE;
369
370 // Added by CS453 Team 6
371 @DoubleValue(min = 0.0, max = 1000.0)
372 public static double DIVERSITY_K = 0.0;
373
374 // Added by CS453 Team 6
375 @DoubleValue(min = 0.0, max = 1000.0)
376 public static double DIVERSITY_CAP = 1.0;
377
378 @Parameter(key = "algorithm", group = "Search Algorithm", description = "Search
379 algorithm")
380 public static Algorithm ALGORITHM = Algorithm.DYNAMOSA;
381
382 // Added by CS453 Team 6
383 private static int operatorBucketsOf(Mutation mutant) {
384     String name = mutant.getMutationName();
385     for (int i = 0; i < MUTATION_OPERATORS.length; i++) {
386         if (name.startsWith(MUTATION_OPERATORS[i])) {
387             return i;
388         }
389     }
390     return MUTATION_OPERATORS.length; // OTHER
391 }
392
393 // Added by CS453 Team 6
394 private double computeDiversityEfficiency(Set<Integer> newKilled) {
395     for (Map.Entry<Integer, MutationTestFitness> entry :
396         this.mutantMap.entrySet()) {
397         Integer id = entry.getKey();
398         Integer operatorBucket = operatorBucketsOf(entry.getValue().getMutation());
399         if (operatorBucketsPresent == 0) {
400             Set<Integer> present = new LinkedHashSet<>();
401             operatorBucketsPresent = Math.min(present.size(), 1);
402         }
403         Set<Integer> killed = new LinkedHashSet<>();
404         killed.addAll(newKilled);
405         long[] counts = new long[MUTATION_BUCKETS];
406         long total = 0;
407         for (Integer id : killed) {
408             Integer bucket = mutantTypeCache.get(id);
409             if (bucket != null) {
410                 counts[bucket]++;
411                 total++;
412             }
413         }
414         if (total == 0) {
415             operatorBucketsPresent = 1;
416             this.lastEntropy = 0.0;
417             this.lastEntropyMore = 0.0;
418             return 0.0;
419         }
420     }
421 }
```

Experiment Strategy

- Choose various open-source library to test
- Run with each candidate fitness function, changing parameter k
- See final result to select best-performance fitness



Experiment: Benchmark Selection

- Diverse Type of Computational domains
- Dense nesting of Conditionals, Loops, & Exception—Good for Branch Reachability
- Pure, Deterministic Math Functions—Good for Infection & Propagation

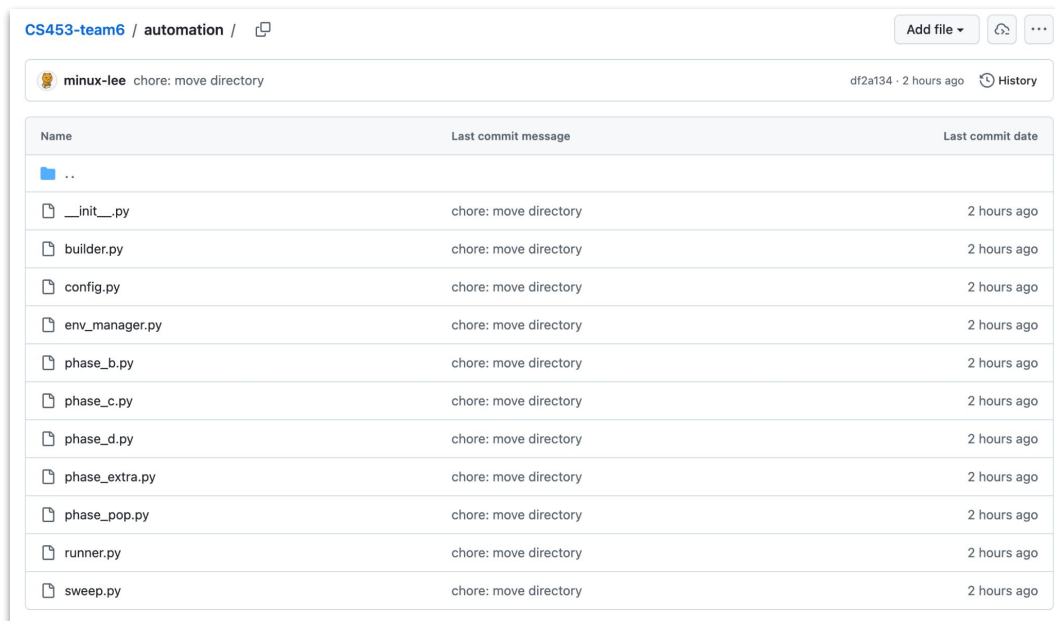


Experiment: Benchmark Selection

Domain	Class	Library	Total Goals
Numerical	ArithmeticUtils	commons-math3	992
	Fraction	commons-math3	709
	Precision	commons-math3	650
	Complex	commons-math3	917
	IntMath	guava	605
	IEEE754rUtils	commons-lang3	176
	NumberUtils	commons-lang3	562
String	CharUtils	commons-lang3	405
	WordUtils	commons-text	827
	Strings	guava	442
	LevenshteinDist.	commons-text	224
Logic	BooleanUtils	commons-lang3	811
Encoding	Hex	commons-codec	379
	Soundex	commons-codec	241

Experiment: Automation

- Had to run through thousands of runs with each taking about a minute
- Wrote automation script in python



The screenshot shows a GitHub repository interface for 'CS453-team6 / automation'. At the top, there's a header with the repository name and a 'Add file' button. Below this, a commit summary for user 'minux-lee' is shown with the message 'chore: move directory' and a commit hash 'df2a134' from '2 hours ago'. A 'History' link is also present. The main content is a table listing files and their commit history.

Name	Last commit message	Last commit date
..		
__init__.py	chore: move directory	2 hours ago
builder.py	chore: move directory	2 hours ago
config.py	chore: move directory	2 hours ago
env_manager.py	chore: move directory	2 hours ago
phase_b.py	chore: move directory	2 hours ago
phase_c.py	chore: move directory	2 hours ago
phase_d.py	chore: move directory	2 hours ago
phase_extra.py	chore: move directory	2 hours ago
phase_pop.py	chore: move directory	2 hours ago
runner.py	chore: move directory	2 hours ago
sweep.py	chore: move directory	2 hours ago

Experiment: Automation

- Challenge: Computational Limitations
 - 14 libraries, 4 fitness functions, multiple seeds
 - Exhaustive search over fine-grained parameter k was impossible
- Multi-Phase Approach
 - Phase A: Preliminary Screening
 - Evaluated all libraries with fewer seeds
 - Explored a wide parameter range using a coarse-grained grid
 - Phase B: Validation
 - Selected promising configurations from Phase A
 - Test with an increased number of seeds
 - Phase C/D: TBD



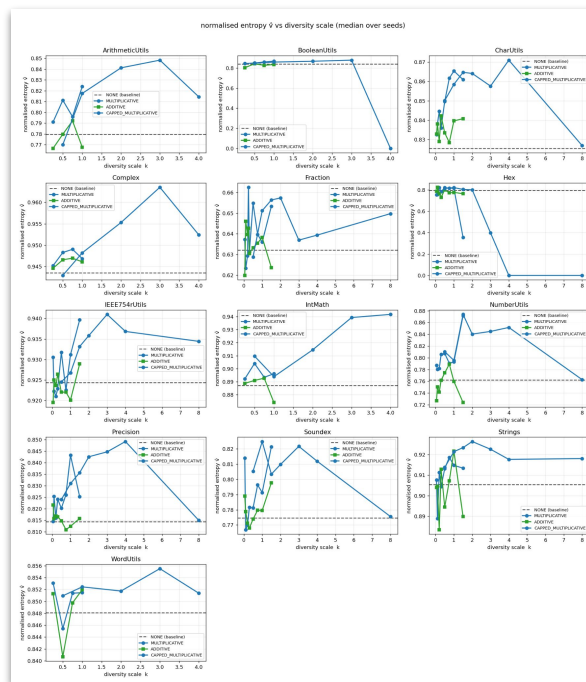
Analysis on Phase A/B

- Plotted results using Python

CS453-team6 / analysis /

minux-lee chore: move directory df2a134 · 3 hours ago History

Name	Last commit message	Last commit date
..		
report	chore: move directory	3 hours ago
init.py	chore: move directory	3 hours ago
honest_audit.py	chore: move directory	3 hours ago
metrics.py	chore: move directory	3 hours ago
multi_phase.py	chore: move directory	3 hours ago
parser.py	chore: move directory	3 hours ago
plots.py	chore: move directory	3 hours ago
slide_figures.py	chore: move directory	3 hours ago
slide_fixed_k.py	chore: move directory	3 hours ago



Analysis on Phase A/B: Fitness Functions

- Exponential: Fails to train properly, excluded for phase C/D
- Additive: Stable & promising performance with k in 0-1.5
- Multiplicative:
 - Good performance with k in 0-1.5
 - But completely collapsed when $k > 2$
 - Hypothesis: Overfitting occurred due to overemphasizing low diversity values



Proposed Solution:

- Adding Capped-Multiplicative mode

Mode	Formula
NONE	F_{base}
MULTIPLICATIVE	$F_{\text{base}} \cdot (1 + k\delta)$
ADDITIVE	$F_{\text{base}} + k\delta$
EXPONENTIAL	$F_{\text{base}} \cdot e^{k\delta}$
CAPPED_MULTIPLICATIVE	$F_{\text{base}} \cdot (1 + \min(k\delta, c))$

Analysis on Phase A/B: Computing Budget

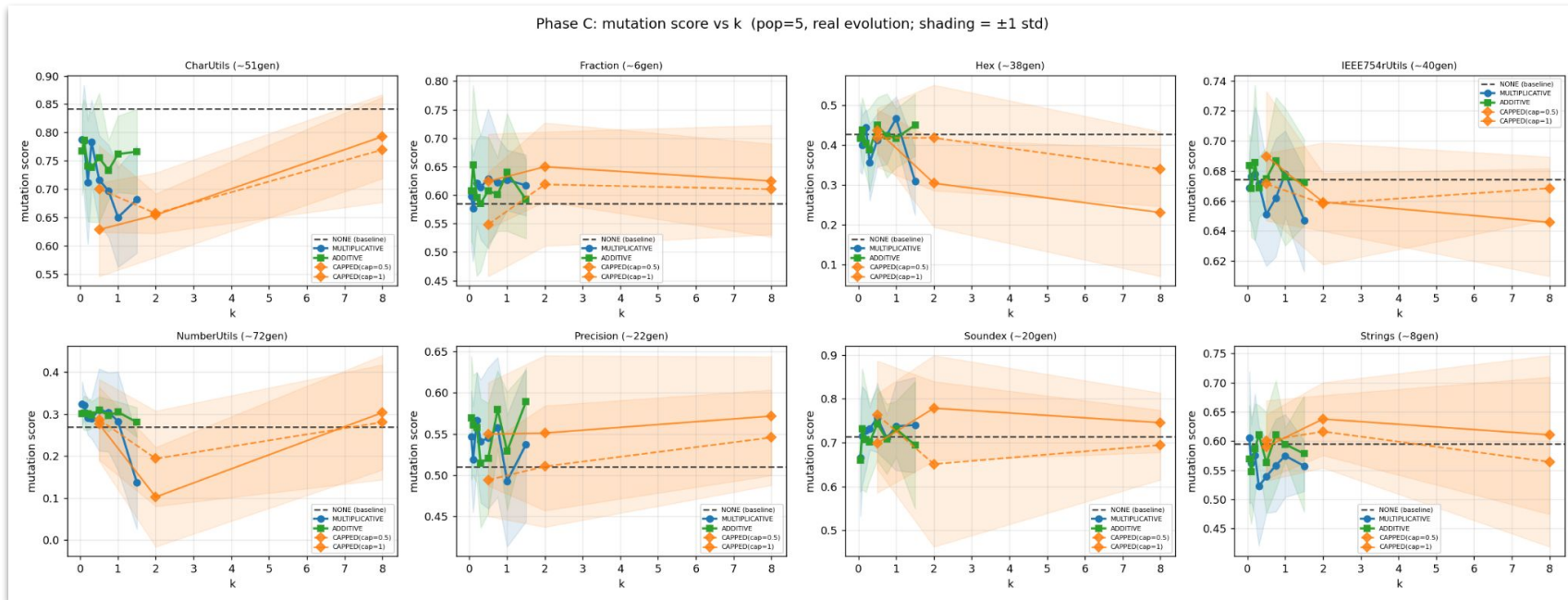
- Most large classes exhausted the time budget before any evolution
 - Populations were set to 50 by default
- Phase C/D switched to population=5, yielding real evolution
 - ~35 generations on average
- Larger class needs more budgets, smaller classes needs less.
 - Phase C for small classes
 - Phase D for large classes



Running Phase C/D: Dense Hyperparameter k

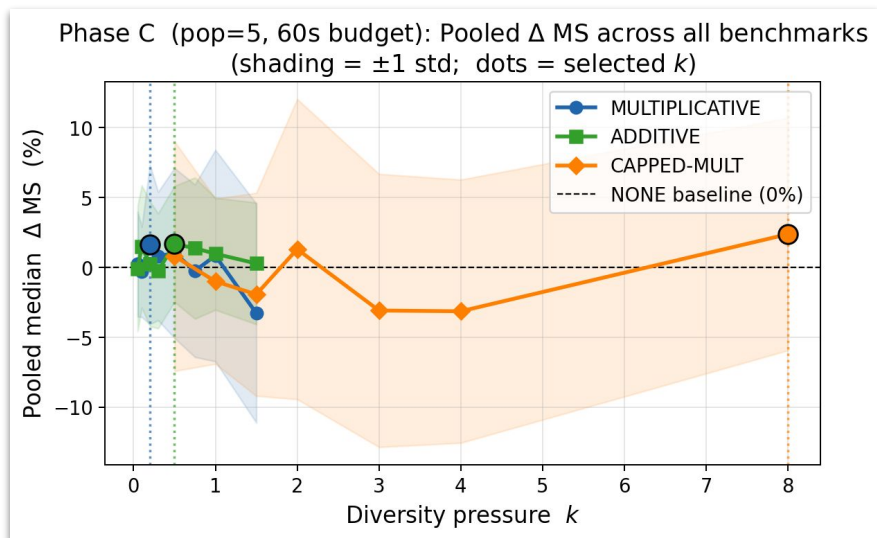
Phase	Benchmarks	Coupling	k values	Seeds	Budget	Pop. size
Phase A (explore)	14	4 modes	$\{0.25 \dots 8\}$ (6)	3	40s	50
Phase B (confirm)	14	Top-5	$\{0.25 \dots 8\}$	10	40s	50
Phase C (refine)	8	4+CAPPED	$\{0.05 \dots 1.5\}$ (8)	8	60s	5
Phase D (large)	6	4+CAPPED	$\{0.25 \dots 4\}$ (5)	6	120s	5
Total						$\approx 3,790$ runs

- Plotted results using Python

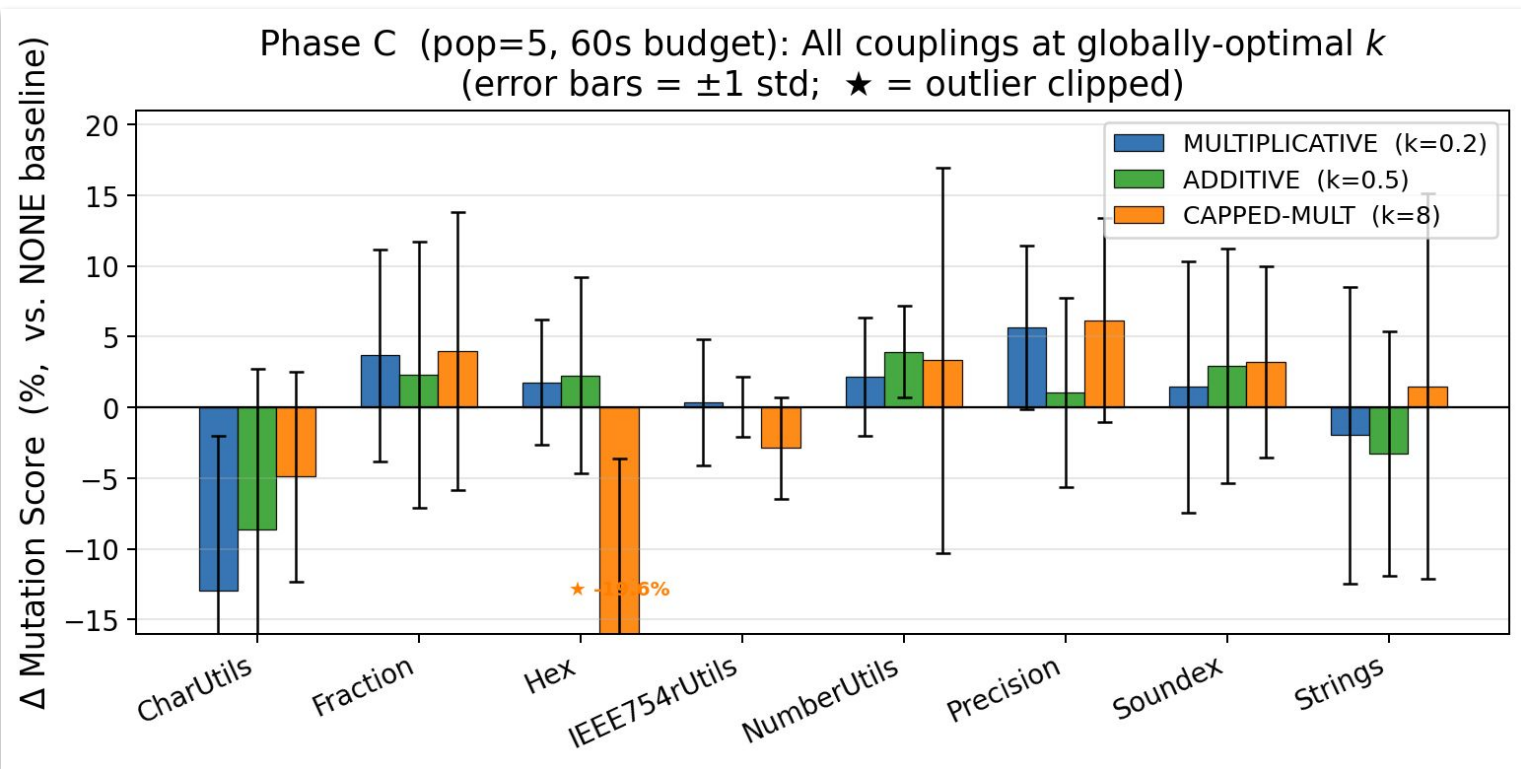


Analysis on Phase C

- Select best- k for each fitness functions, based on average mutation score
 - Additive: $k = 0.5$
 - Multiplicative: $k = 0.2$
 - CAPPED-Multiplicative: $k = 8$

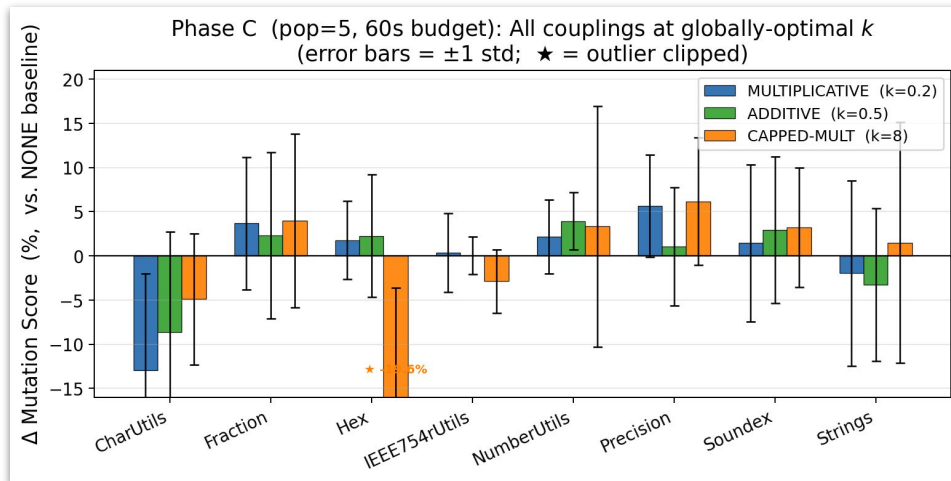


Analysis on Phase C: Mutation Score



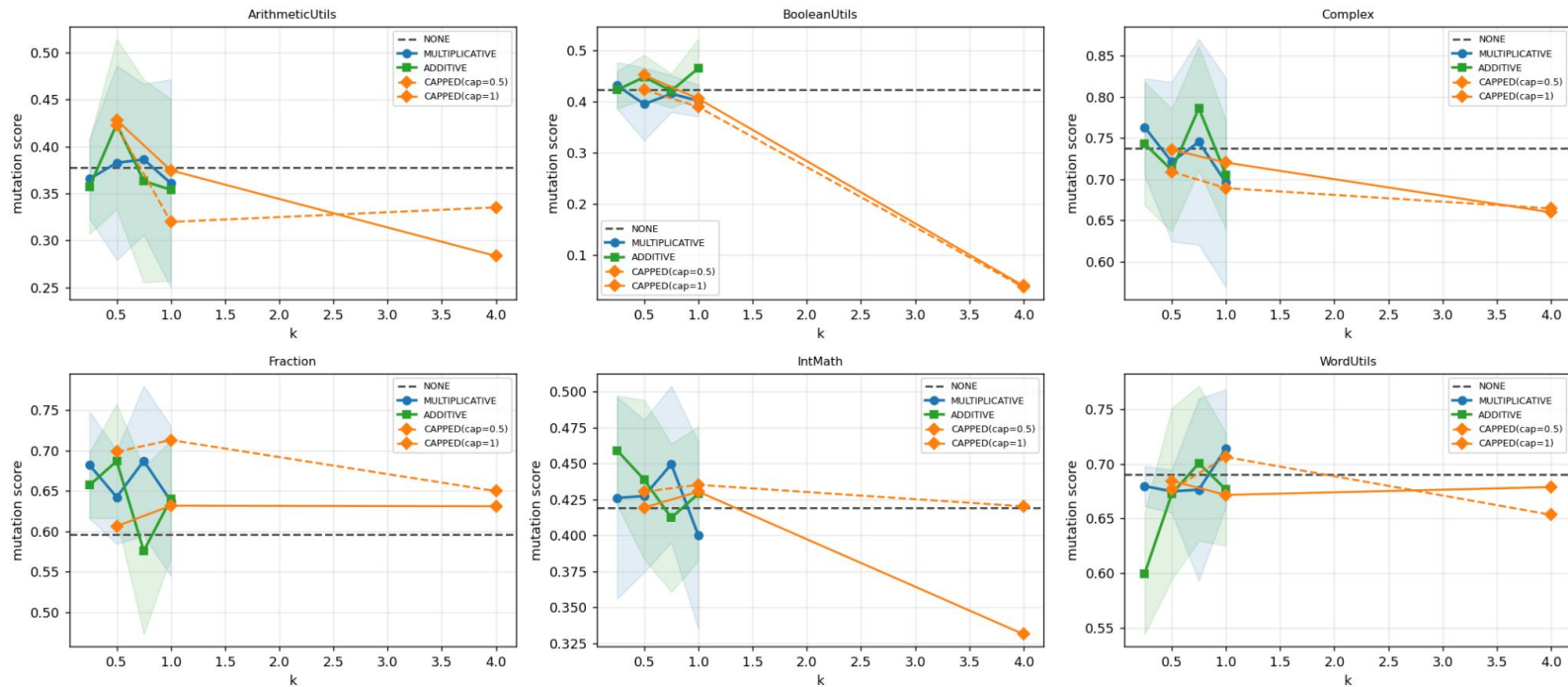
Phase C: CharUtils

- Exceptionally bad result on CharUtils
- Baseline already yields great mutation score and entropy
 - Trade-off occurred:
 - Entropy-guided fitness might shift selection toward diverse operator types at the cost of fewer killed mutants



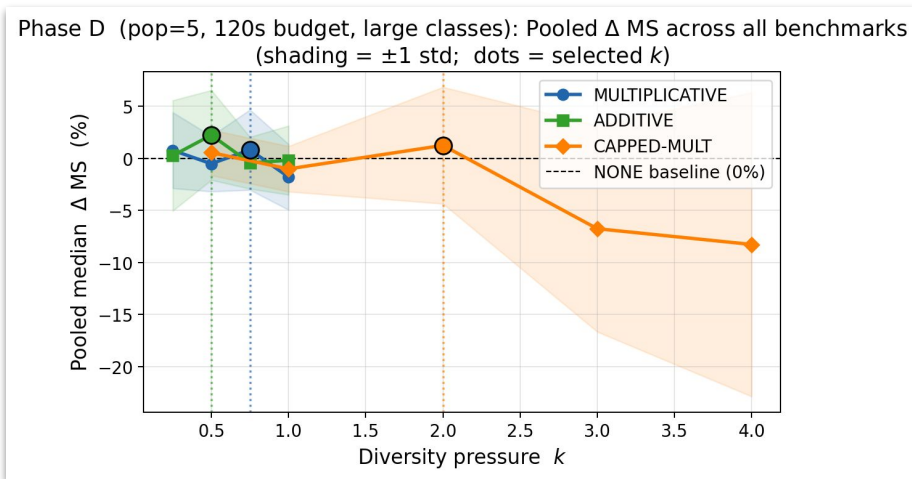
Analysis on Phase D

Phase D: large classes (pop=5, budget=120s, real evolution)

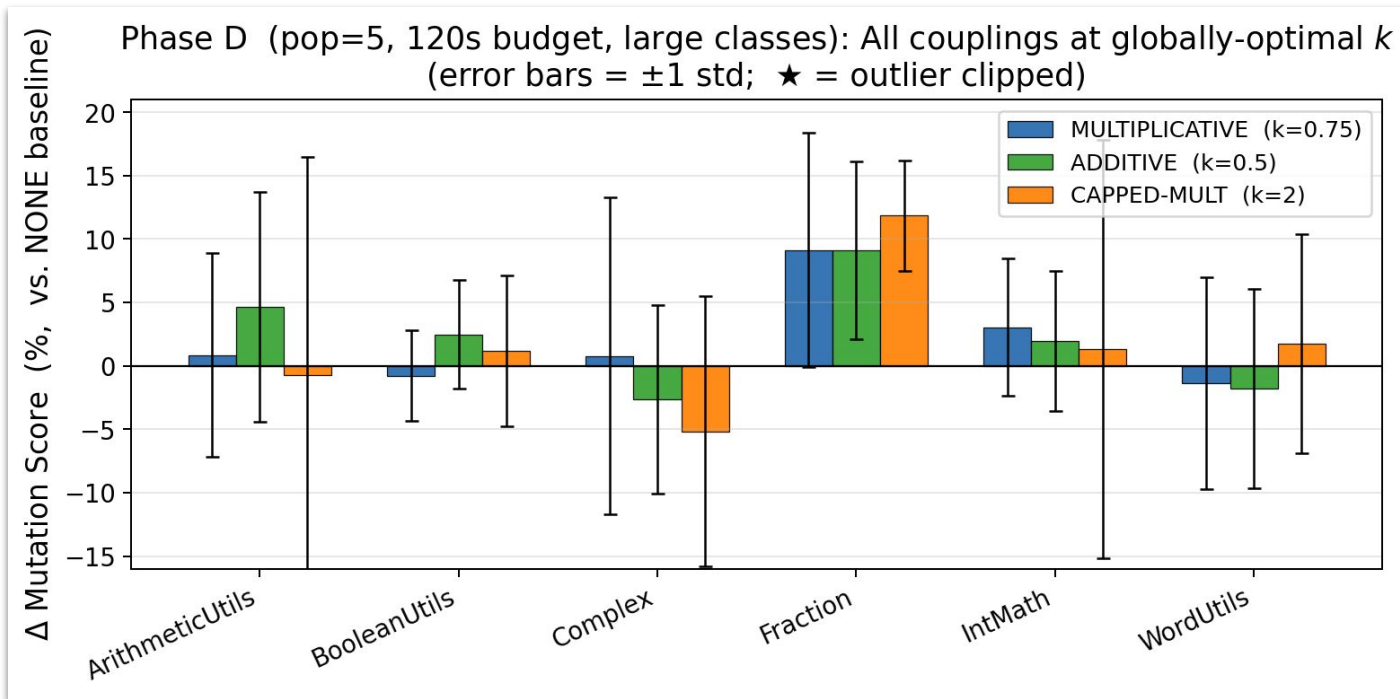


Analysis on Phase D

- Select best- k for each fitness functions, based on average mutation score
 - Additive: $k = 0.5$
 - Multiplicative: $k = 0.75$
 - CAPPED-Multiplicative: $k = 2$

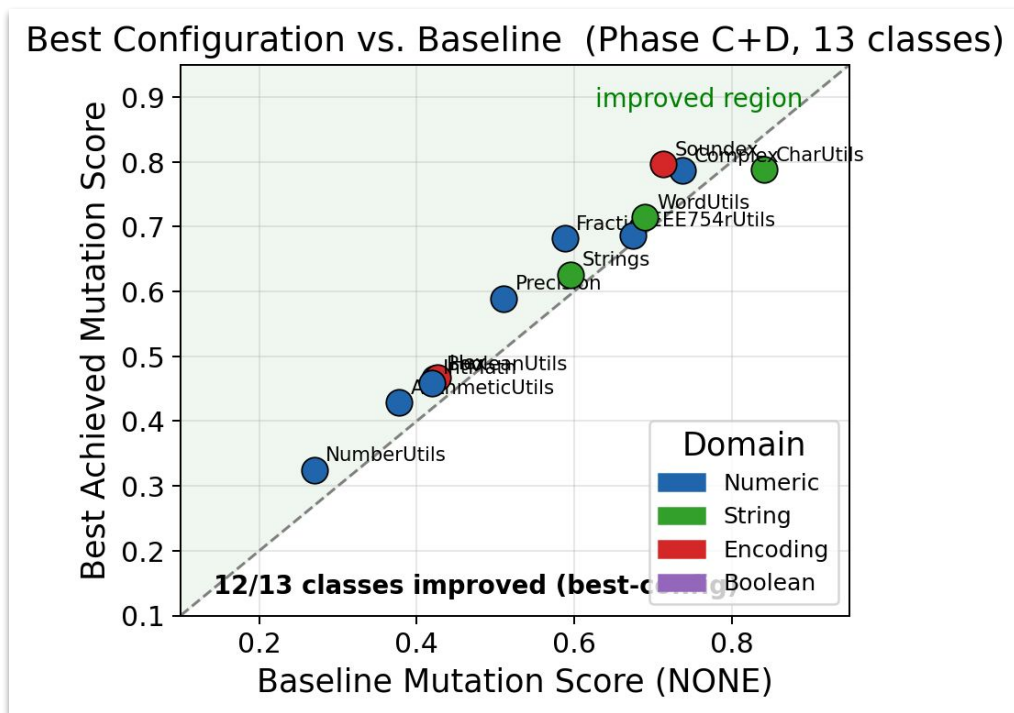


Analysis on Phase D: Mutation score



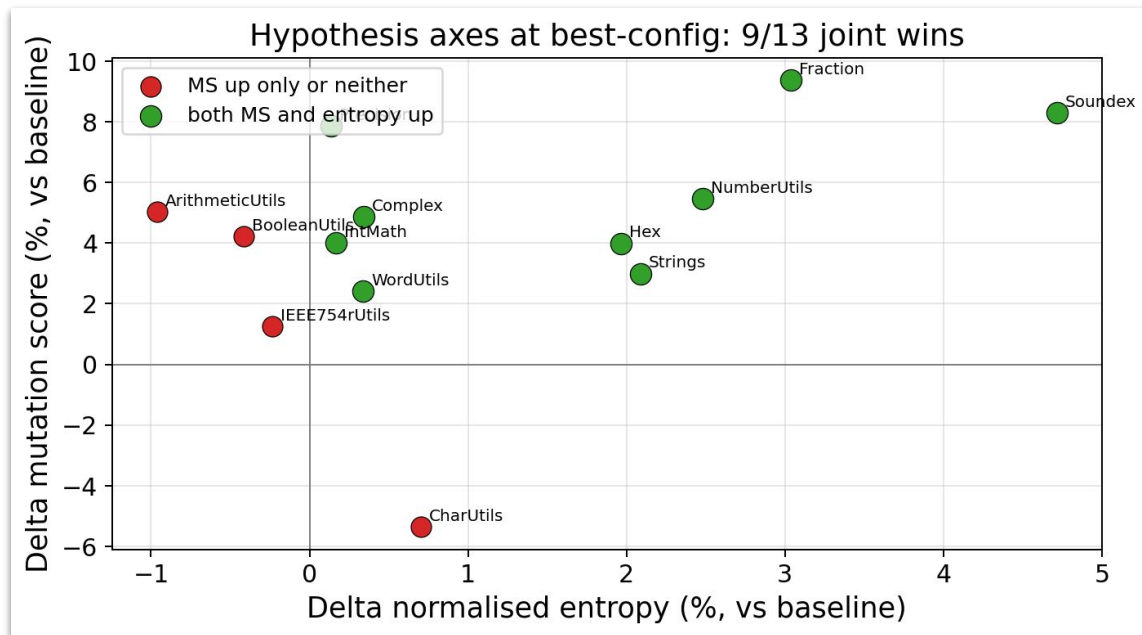
Phase C/D: Summary

- Mutation Score were improved for 12/13 classes under best configuration



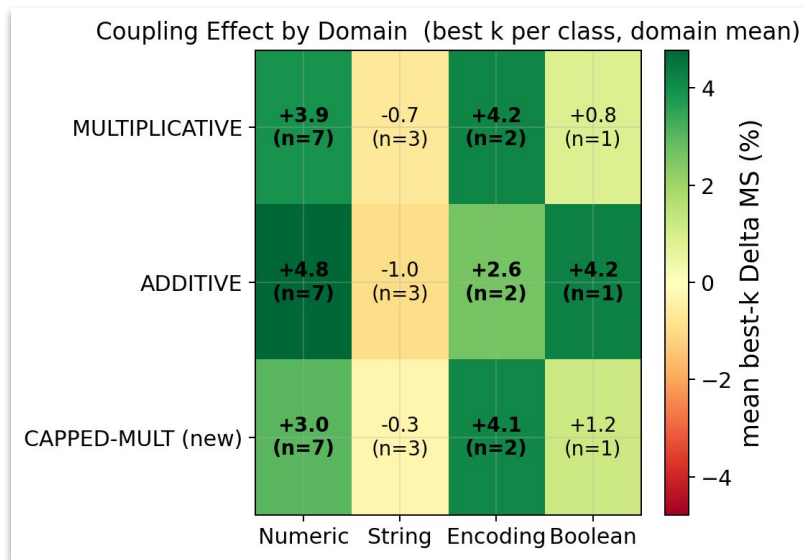
Phase C/D: Correlation between MS and δ

- Both entropy and mutation score were improved for 9/13 classes under best configuration



Phase C/D: Domain Analysis

- Get average Δ mutation score, each for open-source library's domain
- Mutation score was improved in most domains
 - String domain: CharUtils sole exception



Conclusion

- The hypothesis is partially supported:
 - Diversity coupling has potential of improving mutation score on some cases
 - Limitation:
 - Diversity-aware fitness is promising but conditional
 - Can't deploy one formula and one k everywhere
 - Operator entropy is a proxy—real fault detection not tested
 - Future work:
 - Test with larger population, more budgets
 - Track each generations to confirm the hypothesis
 - Learn with entropy at early phase, then learn with pure MS at later phase
 - Systematic evaluation of one global setting
 - CAPPED cap selection
 - etc.
- 

References

McMinn, P. (2004). Search-based software test data generation: a survey. STVR, 14(2), 105–156.

Fraser, G., & Arcuri, A. (2011). EvoSuite: automatic test suite generation for Java classes. ISSTA.

Fraser, G., & Arcuri, A. (2014). Achieving higher test quality with mutation-based test generation. TSE, 40(9), 1041–1059.

Just, R., et al. (2014). Are mutants a valid substitute for real faults? FSE.

Papadakis, M., et al. (2019). Mutation testing advances. Advances in Computers, 112, 1–75.

